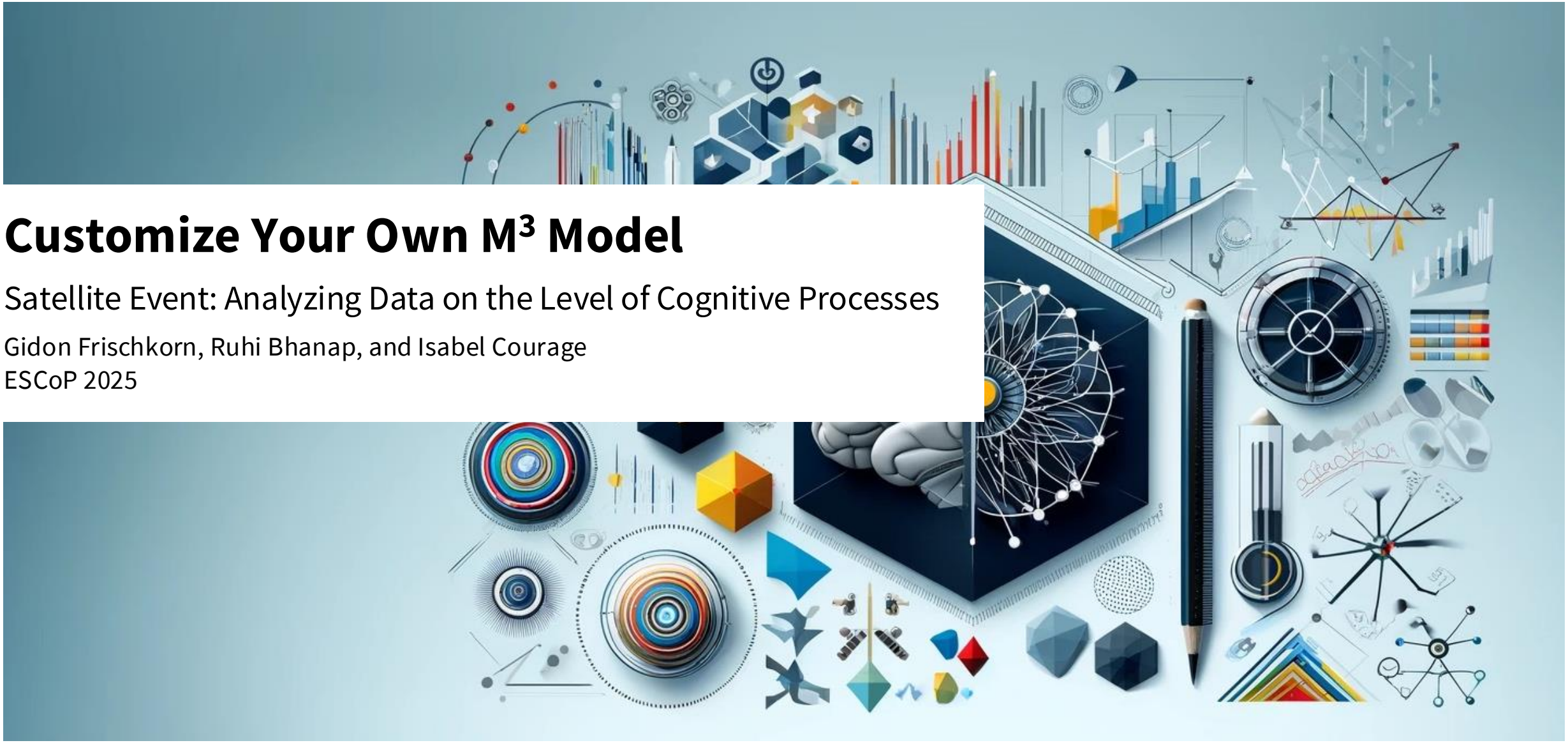
Universität Zürich[UZH]

Department of Psychology



Customize Your Own M³ Model

Satellite Event: Analyzing Data on the Level of Cognitive Processes

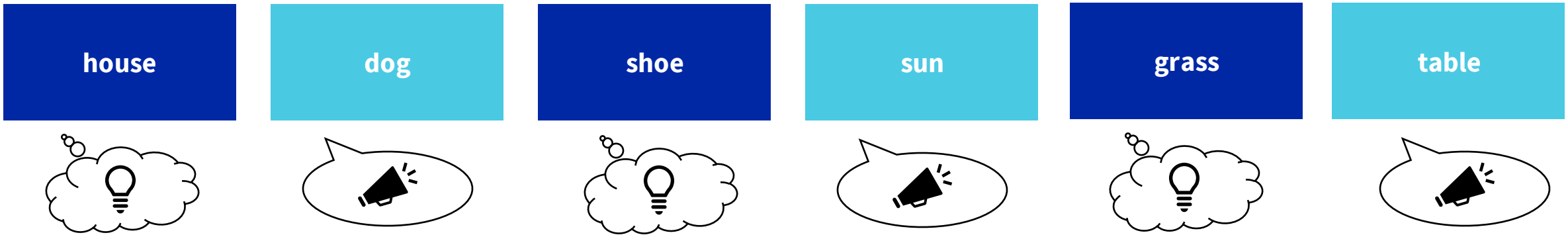
Gidon Frischkorn, Ruhi Bhanap, and Isabel Courage

ESCoP 2025

Extending M³ Models: Complex Span Task

Memory Measurement Model

- Complex Span Task: Memorize the list of items (e.g., words) in the presented + performing an interleaved processing task…

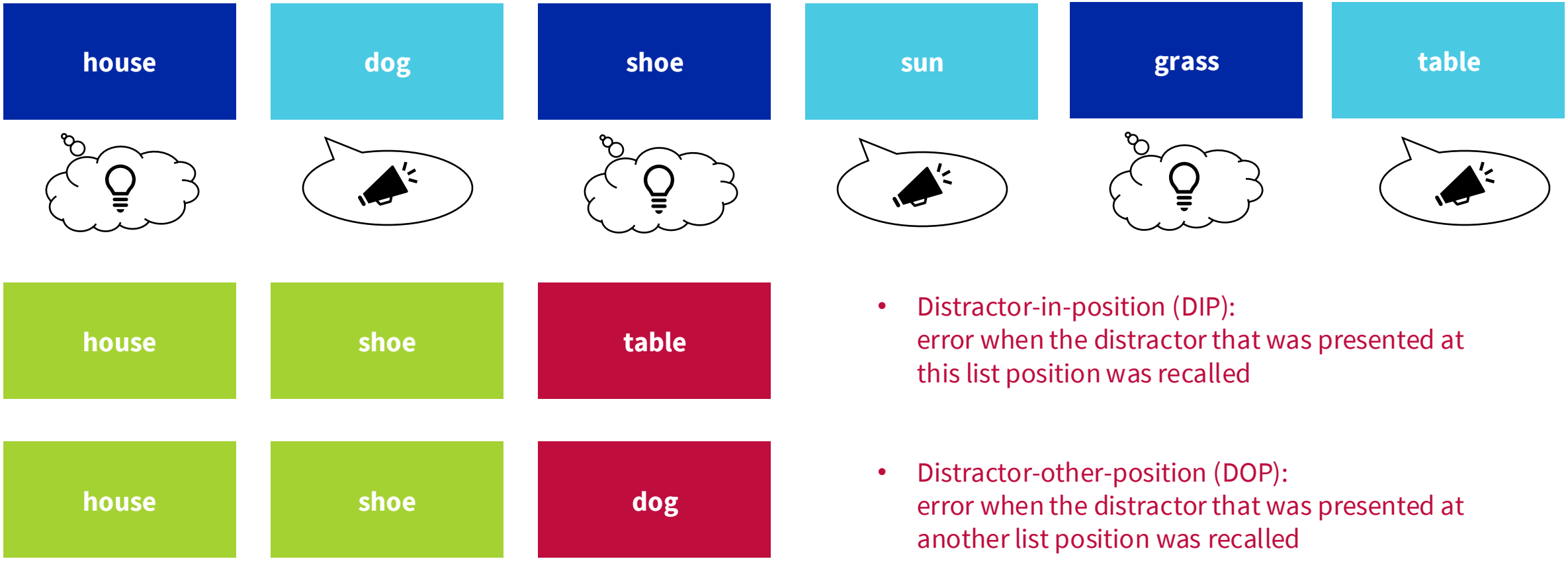


- Please recall the the items in the presented order…

Extending M³ Models: Complex Span Task

Memory Measurement Model

- This extension of the complex span task allow for additional error types:



- Distractor-in-position (DIP):
  error when the distractor that was presented at this list position was recalled

- Distractor-other-position (DOP):
  error when the distractor that was presented at another list position was recalled

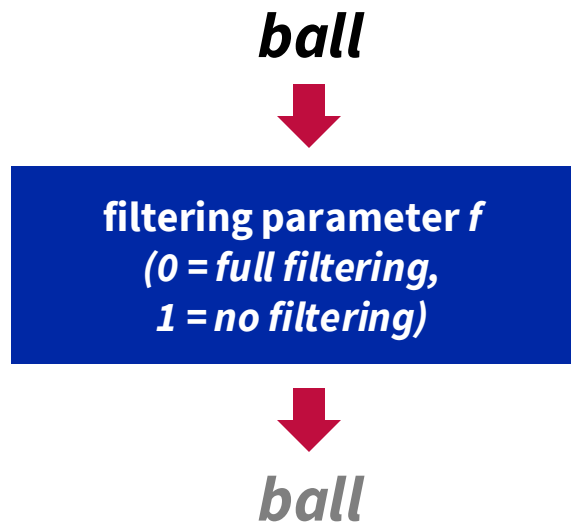
Extending M³ Models: Complex Span Task

Memory Measurement Model

- *Extended model for complex span task* predicts 5 response categories (2 additional categories regarding the distractors):
  - Item-in-position (IIP): correct response
  - Item-other-position (IOP): error when an item that was presented in the list at another list position was recalled
  - Not-presented lure (NPL): error when an item that was not presented in the list at all was recalled
  - Distractor-in-position (DIP): error when the distractor that was presented at this list position was recalled
  - Distractor-other-position (DOP): error when the distractor that was presented at another list position was recalled
  - Extended M³ model for complex span task:

$$A\,(IIP) = b + a + c$$
$$A\,(IOP) = a + c$$
$$A\,(NPL) = c$$
$$A\,(DIP) = b + \boldsymbol{f}\,(a + c)$$
$$A\,(DOP) = b + \boldsymbol{f}\,(a)$$

Available as `version = "cs"` when setting up an `m3()` model



(Oberauer & Lewandowsky, 2019)

Extending M³ Models: Complex Span Task

Memory Measurement Model

- *Extended model for complex span task* predicts 5 response categories
(2 additional categories regarding the distractors):
  - Item-in-position (IIP): correct response
  - Item-other-position (IOP): error when an item that was presented in the list at another list position was recalled
  - Not-presented lure (NPL): error when an item that was not presented in the list at all was recalled
  - Distractor-in-position (DIP): error when the distractor that was presented at this list position was recalled
  - Distractor-other-position (DOP): error when the distractor that was presented at another list position was recalled

- Extended M³ model for complex span task:

$$A\,(IIP) = b + a + c$$

$$A\,(IOP) = a + c$$

$$A\,(NPL) = c$$

$$A\,(DIP) = b + \boldsymbol{ra} * a + \boldsymbol{rc} * c$$

$$A\,(DOP) = b + \boldsymbol{ra} * a$$

Filtering / removal Could Differ for: Item vs binding memory

(Oberauer & Lewandowsky, 2019)

Extending M³ Models: custom models

Memory Measurement Model

Response Categories informative for relevant processes

Data

Custom M3 Models

m3() model object → version = "custom"

Names of Response Categories Match Activation functions

Activation Functions

Linear Model Formula → predict parameters

```
model_formula <- bmmformula(
  c ~ 1 + cond + (1 + cond | ID),
  a ~ 1 + (1 | ID)
  rc ~ 1 + cond + (1 + cond | ID),
  ra ~ 1 + (1 | ID)
)
```

```
act_functions <- bmmformula(
  iip ~ b + a + c,
  iop ~ b + a,
  dip ~ b + ra * a + rc * c,
  dop ~ b + ra * a,
  npl ~ b
)
```

All activation Functions should at least contain background noise b

All other parameter = 0 → Random guessing

Extending M³ Models: custom models

Example: `oberauer_lewandowsky_2019_e1` data → contained in bmm

Memory Measurement Model

```
data <- oberauer_lewandowsky_2019_e1
head(data)
```

```
#> # A tibble: 6 × 10
#>      ID cond             corr other   npl  dist n_corr n_other n_dist n_npl
#>   <int> <fct>           <dbl> <dbl> <dbl> <dbl>  <dbl>   <dbl>  <dbl> <dbl>
#> 1     1 new distractors    78    11     3     8      1       4      5     5
#> 2     1 old reordered      73    23     4    NA      1       4      0    10
#> 3     1 old same           95     4     1    NA      1       4      0    10
#> 4     2 new distractors    49    23     1    27      1
#> 5     2 old reordered      54    44     7    NA      1
#> 6     2 old same           64    23     8    NA      1
```

Custom M3 Models

```
my_model <- m3(resp_cats = c("corr","other","dist","npl"),
               num_options = c("n_corr","n_other","n_dist","n_npl"),
               choice_rule = "simple",
               version = "custom")
```

Data

m3() model object → version = "custom"

Fit the model

```
m3_fit <- bmm(
  formula = full_formula,
  data = data,
  model = my_model,
  sample_prior = "yes",
  cores = 4,
  warmup = 1000, iter = 2000,
  backend = 'cmdstanr',
  file = "assets/bmmfit_m3_vignette",
  refresh = 0
)
```

```
par_formulas <- bmf(
  c ~ 1 + cond + (1 + cond | ID),
  a ~ 1 + cond + (1 + cond | ID),
  d ~ 1 + (1 | ID)
)
```

Activation Functions

Linear Model Formula → predict parameters

```
act_formulas <- bmf(
  corr ~ b + a + c,
  other ~ b + a,
  dist ~ b + d,
  npl ~ b
)
```

```
full_formula <- act_formulas + par_formulas

# pass all formulas in one call
full_formula <- bmf(
  corr ~ b + a + c,
  other ~ b + a,
  dist ~ b + d,
  npl ~ b,
  c ~ 1 + cond + (1 + cond || ID),
  a ~ 1 + cond + (1 + cond || ID),
  d ~ 1 + (1 || ID)
)
```

What are your questions so far?

Evaluating Parameter Recovery

Getting insights how to design your task for a custom M3 model?

Memory Measurement Model

Planning a study with custom M3 models

Task Design

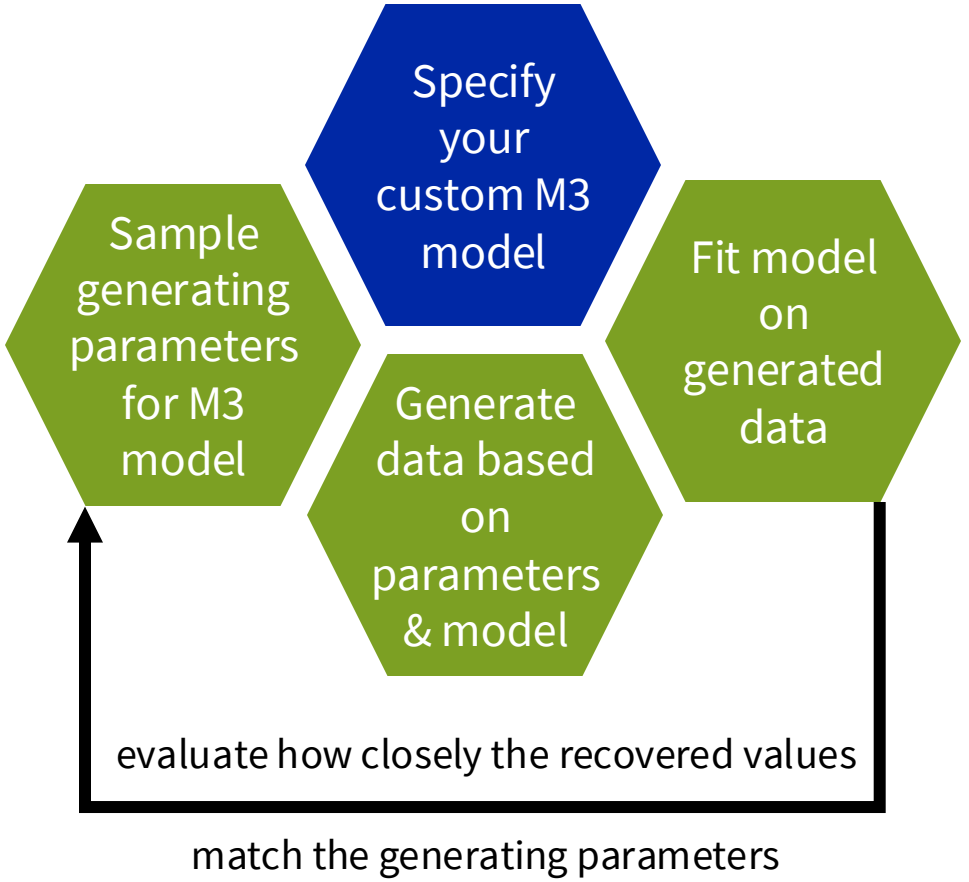
Type of response categories

Difficulty → Expected rate of errors

Data requirements

Number of trials / retrievals in each condition

Number of participants



Evaluating Parameter Recovery

Getting insights how to design your task for a custom M3 model?

Memory Measurement Model

```
custom_m3 <- m3(
  resp_cats <- c("iip", "iop", "dip", "dop", "npl"),
  num_options <- c(1,4,1,4,5),
  choice_rule <- "softmax",
  version = "custom"
)
```

Specify your custom M3 model

```
act_functions <- bmmformula(
  iip ~ b + a + c,
  iop ~ b + a,
  dip ~ b + ra * a + rc * c,
  dop ~ b + ra * a,
  npl ~ b
)
```

Sample generating parameters for M3 model

```
n_subjects <- 30
gen_pars <- data.frame(
  c = rnorm(n_subjects, mean = 3, sd = 0.5),
  a = rnorm(n_subjects, mean = 1.5, sd = 0.4),
  rc = runif(n_subjects, min = 0.2, max = 0.3),
  ra = runif(n_subjects, min = 0.5, max = 0.9)
)
```

Random Generation Functions rbmmodel()

Generate data based on parameters & model

```
n_trials <- 100
sim_data <- rm3(n = 1, size = n_trials,
                pars = gen_pars[i,],
                m3_model = custom_m3,
                act_funs = act_functions)
```

Fit model on generated data

```
# fit the m3 to the simulate data
par_formula <- bmf(
  c ~ 1 + (1 | ID),
  a ~ 1 + (1 | ID),
  rc ~ 1 + (1 | ID),
  ra ~ 1 + (1 | ID)
)

# fit the model with bmm
m3_fit_custom <- bmm(
  formula = act_funs + par_formula,
  model = m3_model_custom,
  data = sim_data
)
```

Extract fixed and random effect → calculate Subject parameters

fixef()

Fixed effects summary

ranef()

Random effects summary

What are your questions so far?

Exercise III: Fitting the M3 for Simple Span tasks

Data from Oberauer (2019)

Task 1: Simple Span task + large word pool

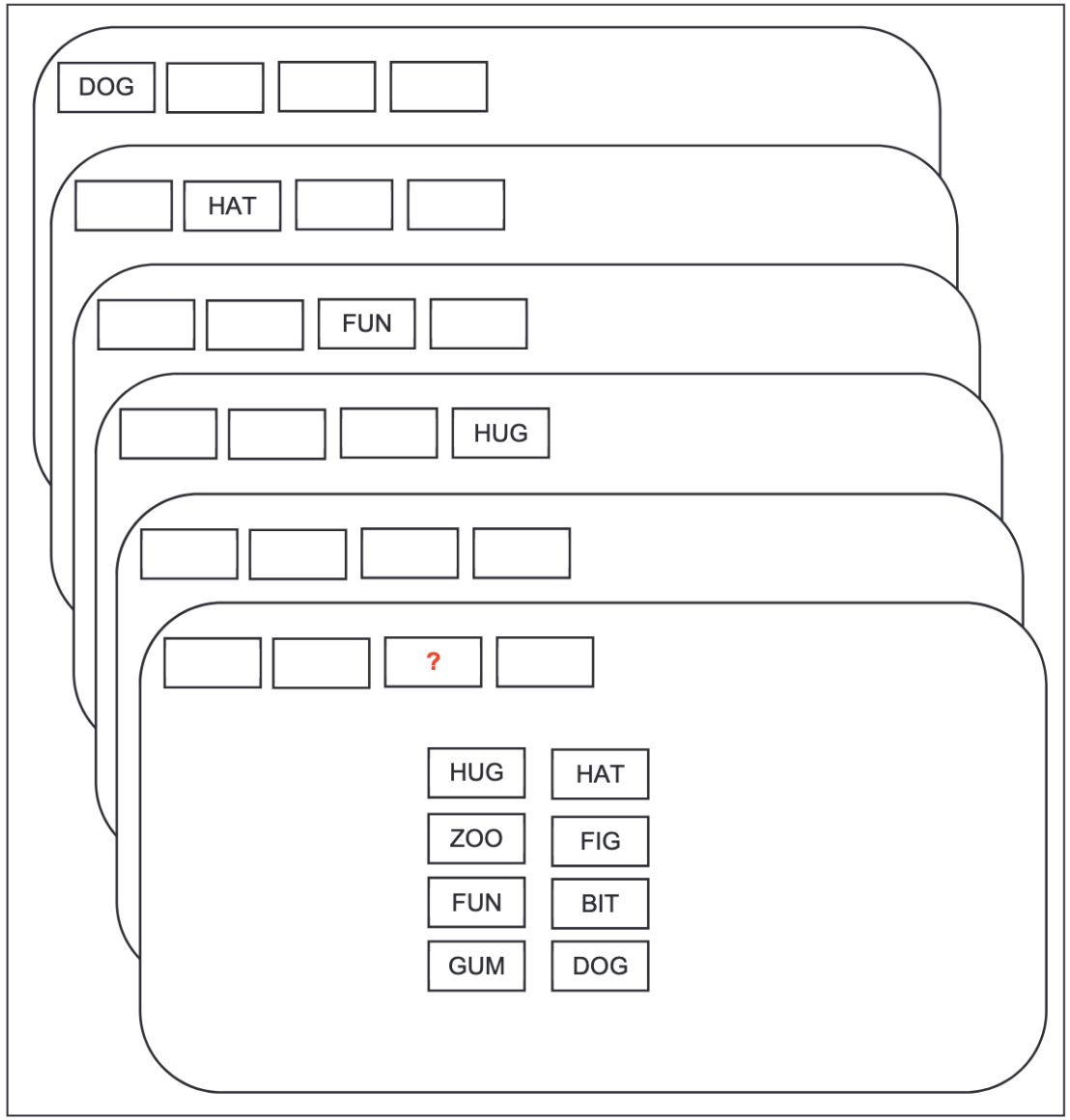
Task 2: Simple Span task + small word pool

Which memory process vary with setsize?

Your task: Implement M3 for simple span task

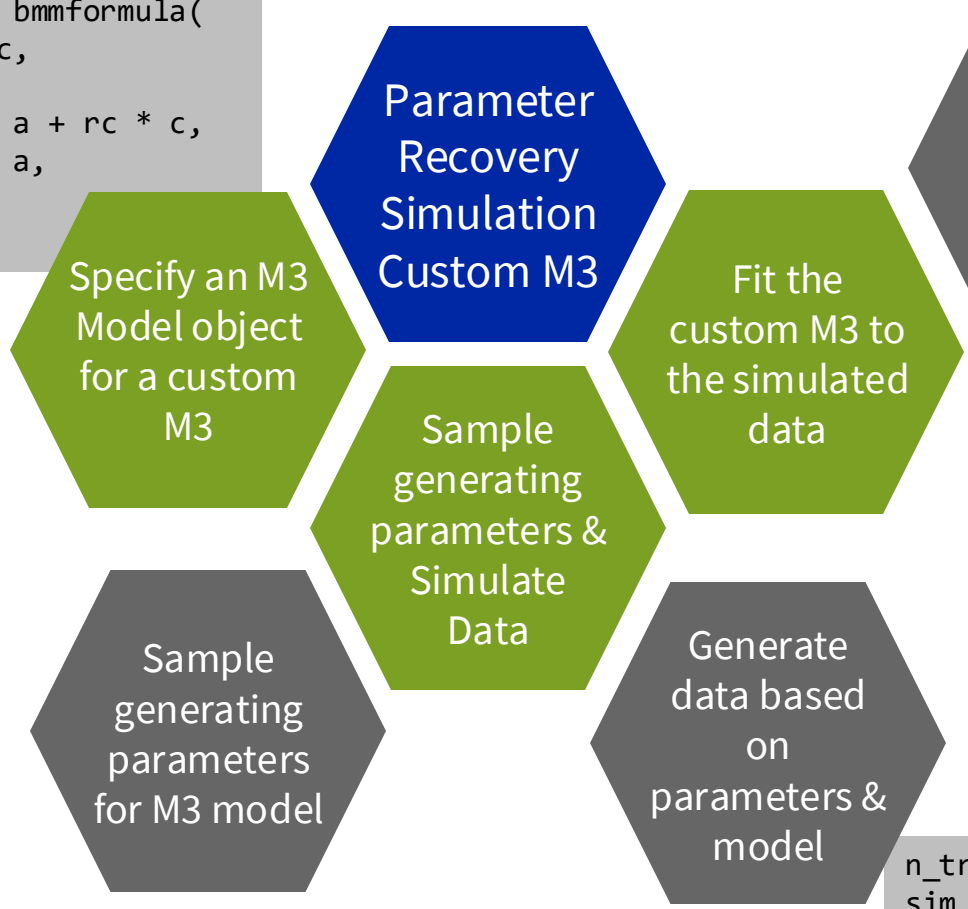
1) Test if context activation varies by set size

2) Test if item activation varies by set size



Exercise IV: Parameter Recovery with custom M3

```
act_functions <- bmmformula(
  iip ~ b + a + c,
  iop ~ b + a,
  dip ~ b + ra * a + rc * c,
  dop ~ b + ra * a,
  npl ~ b
)
```




```
# fit the m3 to the simulate data
par_formula <- bmf(
  c ~ 1 + (1 | ID),
  a ~ 1 + (1 | ID),
  rc ~ 1 + (1 | ID),
  ra ~ 1 + (1 | ID)
)

# fit the model with bmm
m3_fit_custom <- bmm(
  formula = act_funs + par_formula,
  model = m3_model_custom,
  data = sim_data
)
```

```
n_trials <- 100
sim_data <- rm3(n = 1, size = n_trials,
                pars = gen_pars[i,],
                m3_model = custom_m3,
                act_funs = act_functions)
```